MYSTORE ARCHITECTURE GUIDE

How Inertia.js Works and How We Implemented It in This Project

This overview explains the Inertia mental model, the Laravel + React wiring now used in MyStore, and the temporary compatibility layer that lets the migrated UI keep working while we replace old API-style patterns.

Before

A separate React SPA in **frontend/** called Laravel JSON APIs at **/api**.

Now

The app runs from **backend/** with Inertia.js, React, Laravel routes, and server-rendered props.

Current State

A hybrid migration: Inertia handles page visits and auth flow, while some legacy data fetches still use mapped JSON endpoints.

1. What Inertia.js Actually Does

Inertia sits between classic server-rendered apps and a full SPA. Laravel still owns the routes and data loading, but instead of returning Blade pages for every screen, it returns an Inertia page name plus props. React then renders that page component.

Request flow

1. The browser visits a normal Laravel URL like **/profile** or **/seller/products**.
2. Laravel route/controller returns **Inertia::render('App', props)**.
3. Inertia serializes the props into the page response.
4. React receives those props and renders the page without a separate API bootstrap call.
5. Future navigations use Inertia links or router actions, so the page updates without a full reload.

Why teams like it

- No need to maintain a separate SPA router and REST bootstrap layer for every page.
- Auth, redirects, validation errors, and flash messages stay in Laravel where they already fit naturally.
- React still builds the UI, but page data comes directly from controllers.
- It is a strong migration path when a project started as an API-driven SPA.

> Inertia is not replacing React here. It is replacing the “React app calls many page-level APIs just to load” pattern.

2. The MyStore Implementation

We moved the runtime center of gravity into Laravel. The root SPA folder was removed, and the active UI now lives under **backend/resources/js**.

LAYER	FILE	ROLE
Blade root	**backend/resources/views/app.blade.php**	Loads Vite assets, CSRF token, and the Inertia mount point.
React entry	**backend/resources/js/app.js**	Boots Inertia and resolves the top-level page component.
Middleware	**backend/app/Http/Middleware/HandleInertiaRequests.php**	Shares auth user, wishlist, and flash messages with every page.
Controller layer	**backend/app/Http/Controllers/PageController.php**	Builds page-specific props by calling existing controllers and returning **Inertia::render()**.
Routes	**backend/routes/web.php**	Defines normal Laravel web URLs for buyer and seller pages.
UI source	**backend/resources/js/frontend/**	Contains the migrated React components and pages that were copied from the old SPA.

3. Code Example: Inertia Client Bootstrapping

The React entry no longer mounts a standalone SPA with its own router root. Instead, it creates an Inertia app and resolves page names returned by Laravel.

BACKEND/RESOURCES/JS/APP.JS

```
import './bootstrap';
import '../css/app.css';
import { createElement } from 'react';
import { createRoot } from 'react-dom/client';
import { createInertiaApp } from '@inertiajs/react';
import AppPage from './Pages/App.jsx';

createInertiaApp({
    resolve: (name) => {
        if (name === 'App') {
            return AppPage;
        }

        throw new Error(`Unknown Inertia page: ${name}`);
    },
    setup({ el, App, props }) {
        createRoot(el).render(createElement(App, props));
    },
    progress: {
        color: '#0f766e',
    },
});
```

> In this project we currently resolve a single Inertia page name, **App**, and that page re-exports the migrated React app shell.

4. Code Example: Laravel Route and Controller

Route layer

MyStore now uses normal Laravel web routes for buyer and seller pages. For example, **/**, **/orders**, and **/seller/products** all point to page controller methods.

BACKEND/ROUTES/WEB.PHP

```
Route::get('/',
[PageController::class,
'home']);
Route::get('/categories',
[PageController::class,
'categories']);
Route::get('/products/{id}'
, [PageController::class,
'product']);

Route::middleware('auth')-
>group(function () {
    Route::get('/orders',
[PageController::class,
'buyerOrders']);
    Route::get('/profile',
[PageController::class,
'profile']);

Route::get('/seller/product
s', [PageController::class,
'sellerProducts']);
});
```

Controller layer

The controller gathers data and sends it directly as page props instead of forcing the page to fetch everything after render.

BACKEND/APP/HTTP/CONTROLLERS/PAGECONTROLLER.PHP

```
public function home(Request $request)
{
    return Inertia::render('App', [
        'products' => $this->responseData(
            app(ProductController::class)-
>index($request)
        ),
        'categories' => $this->responseData(
            app(CategoryController::class)->index()
        ),
        'brands' => $this->responseData(
            app(BrandController::class)->index()
        ),
    ]);
}
```

5. Code Example: Shared Props for Auth and Wishlist

Inertia middleware is one of the biggest wins in this migration. Instead of fetching the current user on every page load, we share it once from Laravel so React can read it with **usePage()**.

BACKEND/APP/HTTP/MIDDLEWARE/HANDLEINERTIAREQUESTS.PHP

```
public function share(Request $request): array
{
    $user = $request->user();

    return [
        ...parent::share($request),
        'auth' => [
            'user' => $user ? [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role,
            ] : null,
        ],
        'wishlist' => $user && $user->role === 'buyer'
            ? Wishlist::where('user_id', $user->id)->latest()->get()
            : [],
        'flash' => [
            'success' => fn () => $request->session()->get('success'),
            'error' => fn () => $request->session()->get('error'),
        ],
    ];
}
```

BACKEND/RESOURCES/JS/FRONTEND/CONTEXT/AUTHCONTEXT.JSX

```
const { props } = usePage();
const user = props.auth?.user ?? null;

router.post('/login', { email, password }, {
  preserveScroll: true,
  onSuccess: (page) => {
    resolve(page.props.auth?.user ?? null);
  },
  onError: (errors) => {
    reject(new Error(errors.email || errors.password || 'Login failed'));
  },
});
```

6. Code Example: Real Page Hydration in React

A migrated page can now consume server props immediately. In **Home.jsx**, the page first reads **props.products**, **props.categories**, and **props.brands** from Inertia.

BACKEND/RESOURCES/JS/FRONTEND/PAGES/HOME.JSX

```
const { props } = usePage();
const [products, setProducts] = useState(props.products || []);
const [categories, setCategories] = useState(props.categories || []);
const [brands, setBrands] = useState(props.brands || []);

useEffect(() => {
  if (props.categories) {
    setCategories(props.categories);
  }
  if (props.brands) {
    setBrands(props.brands);
  }
  if (props.products) {
    setProducts(props.products);
    setLoading(false);
  }
}, [props.brands, props.categories, props.products]);
```

Some fallback fetch logic still exists in a few pages. That is part of the transition state and can be removed page-by-page as we finish the migration.

7. The Hybrid Migration Layer We Added

We did not fully rewrite the old UI in one pass. To keep momentum, we added a compatibility layer so older client code could keep working while page routing, auth, and initial data moved to Inertia.

- **Router shim: backend/resources/js/inertia-router-dom.jsx** mimics key **react-router-dom** APIs using Inertia navigation underneath.
- **Vite alias: backend/vite.config.js** redirects imports of **react-router-dom** to that shim.
- **Fetch remap: backend/resources/js/bootstrap.js** rewrites old **/api** and **http://127.0.0.1:8000/api** calls to **/data**, adds CSRF, and keeps requests same-origin.

BACKEND/RESOURCES/JS/BOOTSTRAP.JS

```
window.fetch = (input, init = {}) => {
    const requestUrl = typeof input === 'string' ? input : input.url;
    const normalizedUrl = requestUrl.replace('http://127.0.0.1:8000/api', '/data');
    const mappedUrl = normalizedUrl.startsWith('/api/')
        ? normalizedUrl.replace('/api/', '/data/')
        : normalizedUrl;

    return originalFetch(mappedUrl, {
        ...init,
        credentials: 'same-origin',
        headers,
    });
};
```

8. Why We No Longer Need a Separate Frontend Folder

In a full Inertia setup, the React UI belongs to the Laravel app. That means the project does not need a separate root-level frontend application just to talk to backend APIs.

- The old top-level **frontend/** folder was retired because it was a standalone SPA shell.
- The live UI now runs through Laravel Vite assets in **backend/resources/js**.
- We still have a folder named **backend/resources/js/frontend**, but that is just where the copied React app currently lives inside Laravel.
- A later cleanup can rename that folder to something clearer like **app** or **ui**.

9. Recommended Next Steps

PRIORITY	NEXT STEP	WHY IT MATTERS
High	Finish converting remaining pages to rely on **usePage()** props first.	Removes duplicate loading logic and makes page loads more predictable.
High	Replace leftover **fetch('/data/...')** page hydration with controller-supplied props.	Moves more logic into the real Inertia model instead of the compatibility layer.
Medium	Rename **backend/resources/js/frontend** to a clearer app folder.	Reduces migration confusion for future contributors.
Medium	Eventually remove the **react-router-dom** shim.	Simplifies the client stack once all pages navigate through direct Inertia patterns.

Prepared for the current MyStore codebase as of the Inertia migration state visible in **backend/routes/web.php**, **backend/resources/js/app.js**, **backend/resources/js/bootstrap.js**, **backend/app/Http/Middleware/HandleInertiaRequests.php**, and **backend/app/Http/Controllers/PageController.php**.